

《智能系统设计》课程项目计划书

一、项目名称

基于 Jetson Xavier NX 的视觉空气吉他交互系统

二、项目背景与目标

本项目拟使用罗技 USB 摄像头采集演奏者视频，部署于 NVIDIA Jetson Xavier NX 开发板的深度学习与计算机视觉算法实时识别双手动作，并将其映射为虚拟吉他的和弦选择和扫弦操作，最终由系统即时发声。

项目目标是在边缘端完成本地实时视觉交互，形成“看得见、听得到、能演奏”的演示系统，体现以下课程相关能力：

- 深度学习模型在嵌入式边缘端的部署与推理；
- 基于手部关键点的视觉感知；
- 基于时间序列的动作识别与状态控制；
- 低延迟音频交互与系统集成。

预期演示效果

演奏者面对摄像头站立或坐姿演奏。系统在屏幕上叠加显示手部骨架、虚拟琴颈、当前和弦、扫弦方向与帧率：

- 左手在虚拟琴颈上的不同区域选择预设和弦；
- 右手划过虚拟琴弦区域时触发扫弦；
- 系统播放对应和弦音效，并可根据扫弦速度调整音量。

为保证课程项目的可完成性，首版不追求逐根琴弦、逐个手指精确模拟真实按弦，而采用“左手选和弦 + 右手扫弦”的交互方式。

三、功能范围

3.1 基础功能

- 摄像头实时采集单人视频；
- 实时检测左右手及其关键点；
- 左手位置映射至少 4 种预设和弦，例如 C、G、Am、F；
- 右手快速横向或纵向动作识别为扫弦；
- 按当前和弦和扫弦方向触发声音；
- 屏幕实时显示摄像头画面、手部骨架、虚拟琴颈、当前和弦和 FPS；
- 加入关键点平滑、触发冷却和连续帧确认，减少误触发。

3.2 可选加分功能

- 根据扫弦速度控制音量或力度；
- 以特定手势切换音色或演奏模式；
- 加入节拍器、伴奏或歌曲演示模式；
- 通过 MIDI 输出控制电脑端音乐软件或外部乐器；
- 演奏录制与回放；
- 使用 TensorRT 对模型进行优化，并展示优化前后的帧率和延迟对比。

四、硬件设备清单

类别	设备	必需性	用途
边缘计算	NVIDIA Jetson Xavier NX 开发板	必需	运行视觉模型、动作判定、音频控制
视觉输入	罗技 USB 摄像头	必需	采集演奏者视频流
显示设备	HDMI 显示器、键盘、鼠标	必需	调试与展示交互界面
音频输出	USB 音箱或 3.5 mm 有源音箱	必需	播放吉他声音
存储	高速 microSD 卡或 NVMe SSD	推荐	存放系统、模型、音频和演示录像
网络	网线或 Wi-Fi	推荐	环境配置、远程调试、代码同步
灯光	台灯或补光灯	推荐	改善弱光、逆光下的手部识别稳定性
交互外设	USB 按钮、LED 灯带	可选	系统启停、状态或节拍反馈

类别	设备	必需性	用途
音乐外设	USB MIDI 设备	可选	扩展 MIDI 控制能力

硬件布置建议

- 摄像头固定于显示器上方或三脚架上，距离演奏者约 0.8～1.5 米；
- 使用均匀的正面补光，避免强逆光；
- 演示时保持单人、双手完整位于画面内，背景尽量简洁；
- 通过有源音箱输出，避免 Jetson 板载音频接口兼容性影响演示。

五、技术路线



5.1 视觉识别模块

首选使用 MediaPipe Hand Landmarker 进行实时双手关键点检测。该模型能够输出左右手标签和每只手的 21 个关键点，可满足本项目对手腕、指尖位置与手部运动轨迹的需求。

建议从 640×480、30 FPS 的摄像头输入开始测试，在单人、良好光照的受控环境下保证稳定性。

5.2 动作识别与交互映射

模块	输入特征	判定方式	输出
左手和弦识别	左手手腕/食指指尖坐标	判断其落入虚拟琴颈的哪个区域	当前和弦
右手扫弦识别	右手手腕或食指连续坐标	判断是否穿过琴弦区域， 以及速度与方向是否超过阈值	上扫/ 下扫事件
音量控制	右手扫弦速度	将速度映射到预设音量档位	音量/力度
防误触发	连续帧状态、事件时间戳	连续帧确认与冷却时间	稳定触发事件

具体规则如下：

1. 左手在一个和弦区域内连续停留若干帧后，更新当前和弦；
2. 右手进入虚拟琴弦区域并在短时间窗口内产生足够大的位移时，识别为一次扫弦；
3. 同一方向扫弦触发后设置约 150 ms 的冷却时间；
4. 对关键点坐标使用滑动平均或指数平滑，降低检测抖动；
5. 仅在左右手状态均有效时触发音乐事件，避免单手误触发。

5.3 音频模块

音频实现采用逐步递进的方式：

1. ****第一阶段：****使用预录制的和弦 WAV 文件，在识别到扫弦事件时直接播放；
2. ****第二阶段：****使用 `pygame`、`sounddevice` 或 `PortAudio` 管理低延迟音频播放；
3. ****加分阶段：****使用 `mido` 和 `python-rtmidi` 输出 MIDI 信号，控制外部音源或音乐软件。

5.4 Jetson 优化方向

首版以功能稳定为第一优先级，后续根据性能测试决定是否优化：

- 控制摄像头输入分辨率与模型检测频率；
- 将视频采集、模型推理和音频播放解耦；
- 使用 GPU 加速视觉推理；
- 在加分版本中尝试 TensorRT 模型部署或推理优化；
- 记录并对比优化前后的 FPS、CPU/GPU 占用和端到端延迟。

六、软件环境

- 操作系统：JetPack 对应 Ubuntu 系统；
- 开发语言：Python 3；
- 计算机视觉：OpenCV；
- 手部关键点模型：MediaPipe；
- GPU/推理加速：CUDA、TensorRT（进阶）；
- 音频：Pygame、SoundDevice 或 PyAudio；
- MIDI：Mido、python-rtmidi（可选）；
- 协作与版本管理：Git + GitHub/Gitee；
- 测试数据记录：CSV 文件记录 FPS、延迟和识别成功率。

七、六周工期规划

周次	核心任务	具体工作	阶段验收产出
第 1 周	环境与硬件验证	配置 Jetson 环境； 测试摄像头、显示器和音箱； 建立代码仓库	Jetson 能显示实时画面并播放测试音频
第 2 周	手部关键点检测	集成 MediaPipe； 实现双手检测、 骨架绘制和关键点记录	实时双手骨架演示程序
第 3 周	左手和弦识别	设计虚拟琴颈和和弦区域； 实现左手区域判定与稳定切换	稳定选择至少 4 个和弦
第 4 周	右手扫弦与发声	实现扫弦判定、音频触发、 平滑、防抖和冷却机制	能连续演奏且误触发可接受
第 5 周	集成与性能优化	完善界面；开展不同距离、 光照、用户条件下测试； 优化性能	稳定演示版及测试数据
第 6 周	加分功能与答辩准备	完成一项加分功能； 录制演示视频；完成报告、 PPT 和现场预案	最终工程、演示视频、答辩材料

若实际周期不足 6 周，应优先保留“手部识别、4 个和弦、扫弦发声、界面反馈”四项核心功能，MIDI、录制和 TensorRT 优化可以作为后续扩展。

八、人员分工建议

以四人小组为例：

成员	主要职责	阶段交付
成员 A	Jetson 环境、摄像头与音频驱动、部署和性能测试	可运行的嵌入式演示环境、性能数据
成员 B	MediaPipe 集成、手部检测、关键点处理与可视化	双手关键点识别模块
成员 C	和弦区域、扫弦动作、状态机、防抖算法	稳定的动作到音乐事件映射模块
成员 D	音频播放、MIDI 扩展、交互界面、视频与答辩材料	音频模块、完整演示界面、答辩资料

建议每周至少进行一次集成测试。不要等各模块全部完成后才联调，因为视觉识别、状态判定与音频延迟之间需要共同调参。

九、验收指标

指标	目标
检测对象	单人、双手
和弦数量	至少 4 种
手部检测成功率	正常室内光照下不低于 90%
系统帧率	不低于 20 FPS
扫弦到发声的主观延迟	小于 200 ms
连续演奏能力	能完成至少 20~30 秒连续演示
部署方式	Jetson 本地运行，不依赖云端推理

十、风险与应对措施

风险	影响	应对措施
手部遮挡、逆光、背景杂乱	关键点检测不稳定	固定机位、补光、简化背景、限定演示区域
右手动作引发频繁误触发	演奏混乱	使用区域限制、速度阈值、连续帧确认和冷却时间
音频延迟或声音叠加	交互体验差	采用短音频，控制并发播放数量，使用异步音频播放
Jetson 软件依赖配置困难	延误开发	先在电脑完成算法验证，再移植到 Jetson；保留基础可运行版本
功能范围过大	无法按时完成	以基础功能为最低交付标准，扩展功能按优先级逐项加入

十一、最终交付物

- Jetson Xavier NX 上可运行的项目源代码；
- 模型、音频资源及部署说明；
- 系统演示视频；
- 项目报告：需求、设计、算法、测试数据、结果分析与不足；
- 答辩 PPT；
- 可现场演示的空气吉他交互系统。

十二、结论

本项目具有明确的应用场景和良好的现场展示效果，技术范围覆盖深度学习、计算机视觉、嵌入式边缘计算与人机交互。采用“左手选和弦 + 右手扫弦”的设计能在可控工作量内实现稳定、可演示的成果；后续可通过 MIDI、节拍器、TensorRT 优化等功能提高项目深度与完成度。